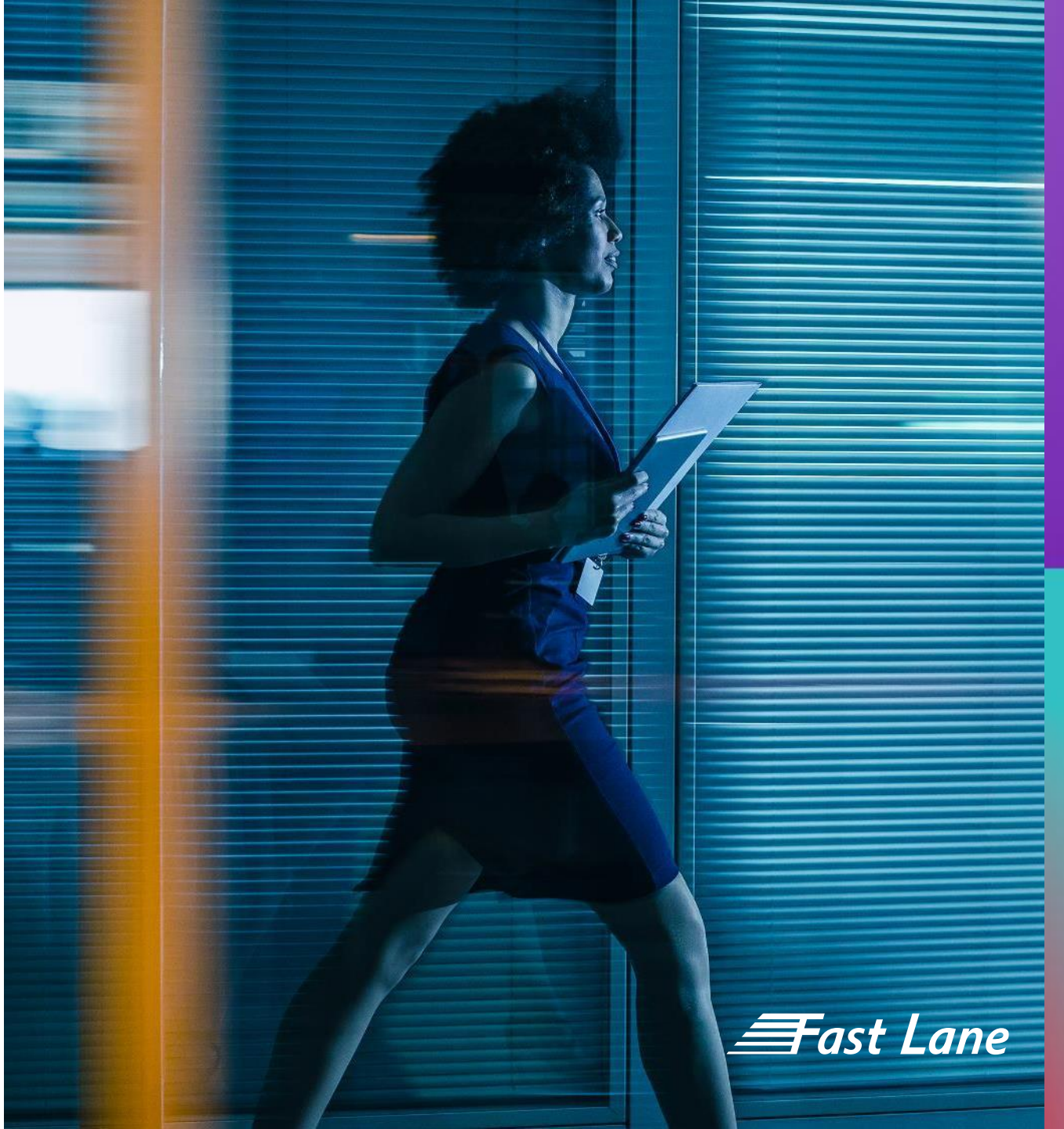
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE — ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT — ★★★

SCAN TO CONNECT
integrations.at

MODULE 6

Scale Task Execution with Sub-Agents and Agent Teams

Specialists in isolated context windows



01

Introduction

Context isolation and agent frontmatter

02

Domain Agents

Backend, frontend and QA specialists

03

Testing Agent Performance

Test cases, assertions and gaps

04

Orchestration Patterns

Delegation, loops and graphs

05

Agent Teams

Lead, teammates, task list and mailbox

06

Parallel Review

Spawn, steer and merge three reviewers

07

Debate and Control

Competing hypotheses, approvals and gates

Scaling work with sub-agents

Specialists in isolated context windows

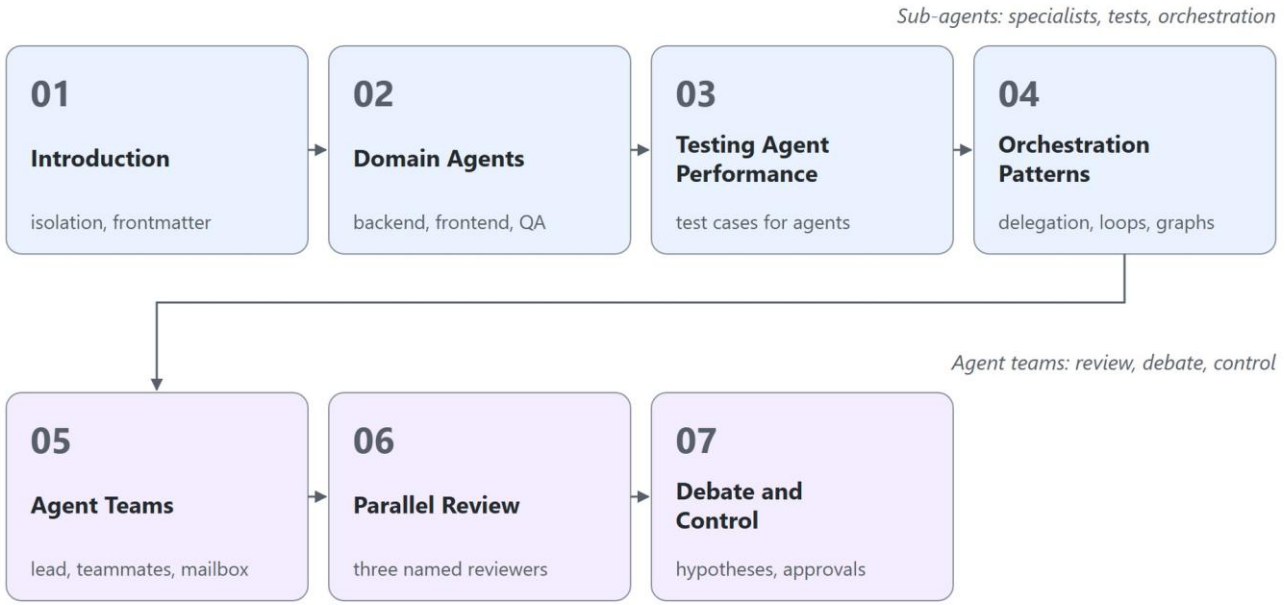
- **Delegation**: an orchestrator hands focused sub-tasks to specialists
- **Isolation**: every sub-agent works in its own clean context window
- **Orchestration**: delegation, loops and graphs, each with its own control flow



Overview

The path through sub-agents and agent teams

- Start with why one agent hits a ceiling and how **frontmatter** shapes a sub-agent
- Build **domain agents** for .NET and Angular with their own skills and MCPs
- Add a **QA agent** that verifies but never writes
- Test an agent like a function: **input, expected output, assertion**
- Orchestrate with **delegation, loops and graphs**
- Finish with **agent teams**: parallel review, debate and control



01

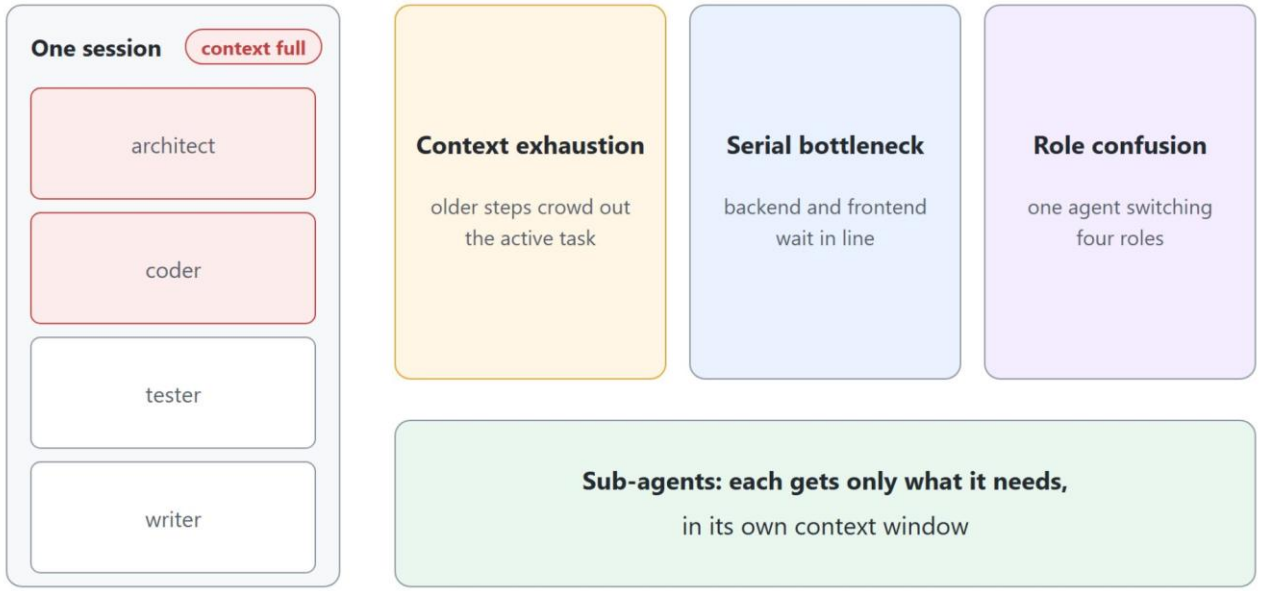
Introduction

Context isolation and agent frontmatter

Why one agent hits a ceiling

Context exhaustion, serial work and role confusion

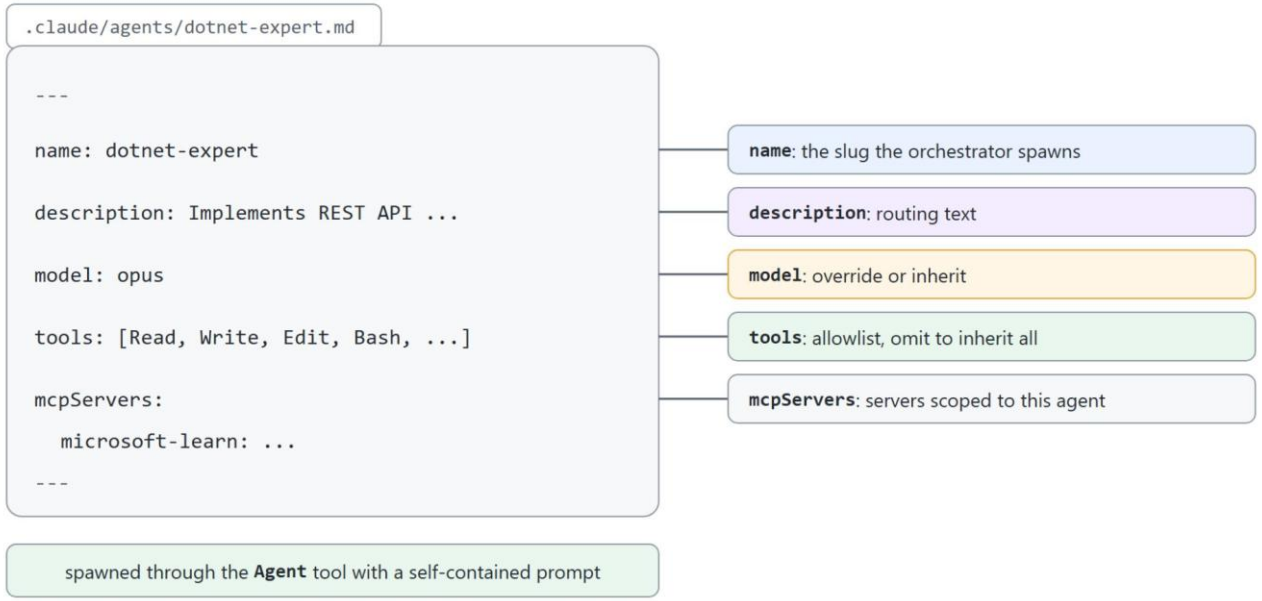
- **Context exhaustion**: every step adds tokens, and older steps cannot be dropped safely
- **Serial bottleneck**: independent work such as code and tests waits in line
- **Role confusion**: one agent as architect, coder, tester and writer is less precise
- A bigger window does not help: earlier steps still compete with the active task



Sub-agent frontmatter

Identity, model, tools and permission mode in one YAML block

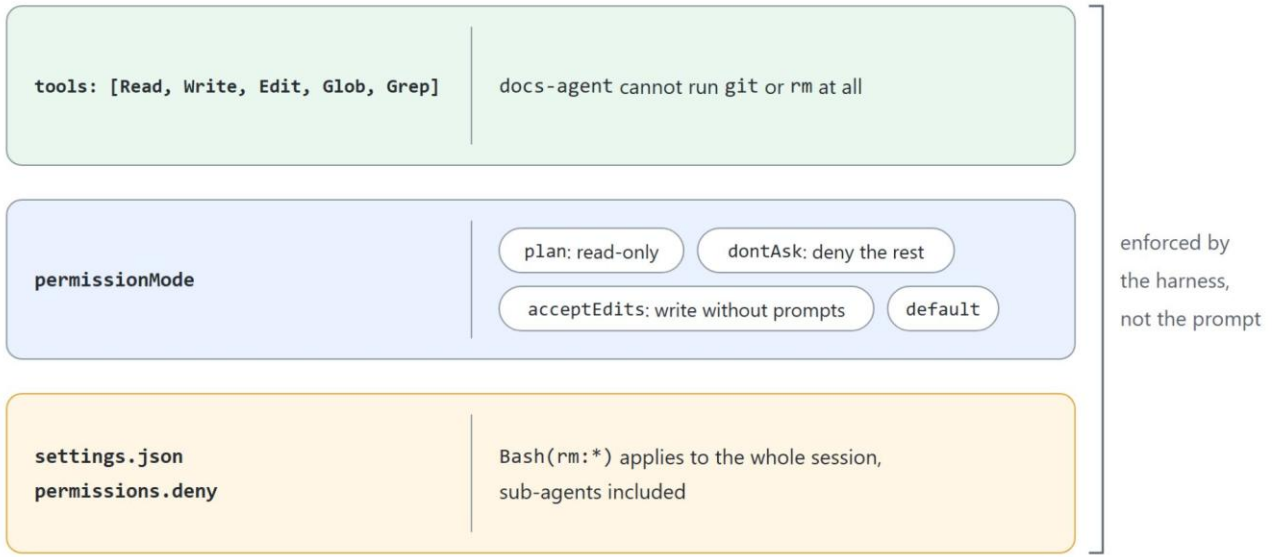
- A Markdown file in `.claude/agents/`, spawned through the `Agent` tool
- `name` and `description` are required; the description is the routing text
- `model` overrides the session model; omit it to inherit
- `tools` is an allowlist, `disallowedTools` a denylist applied first
- `mcpServers` scopes MCP servers to this one agent



Restricting tools and permissions

What an agent cannot call, it cannot misuse

- A docs agent without **`Bash`** cannot run `git` or `rm`, whatever its prompt says
- **`permissionMode`**: `plan` is read-only, `dontAsk` denies what is not allowed
- **`acceptEdits`** lets an implementation agent write without prompting
- Command rules such as `Bash(rm:*)` live in **`settings.json`** and apply to every sub-agent



02

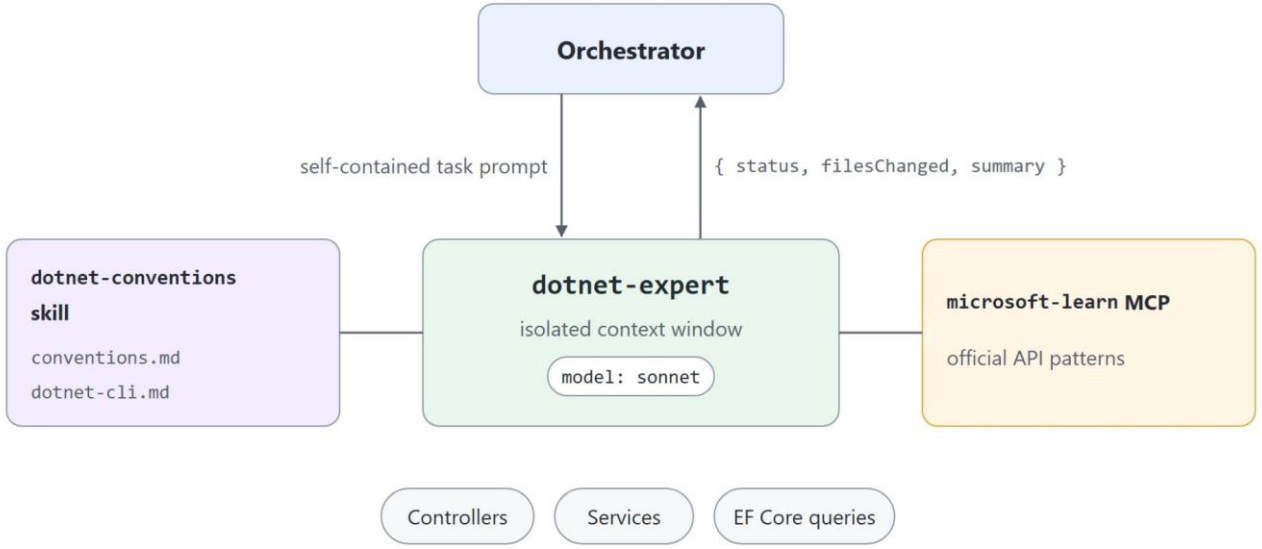
Domain Agents

Backend, frontend and QA specialists

A backend agent for .NET

dotnet-expert with its skill and Microsoft Learn MCP

- Scoped to .NET only: controllers, services, repositories and EF Core
- Applies the `dotnet-conventions` skill on every file it touches
- Checks API patterns against official docs through `microsoft-learn` MCP
- Returns a typed JSON result when the orchestrator asks for one
- No frontend or test artifacts reach its context



Delegating a backend task

No open decisions left in the prompt

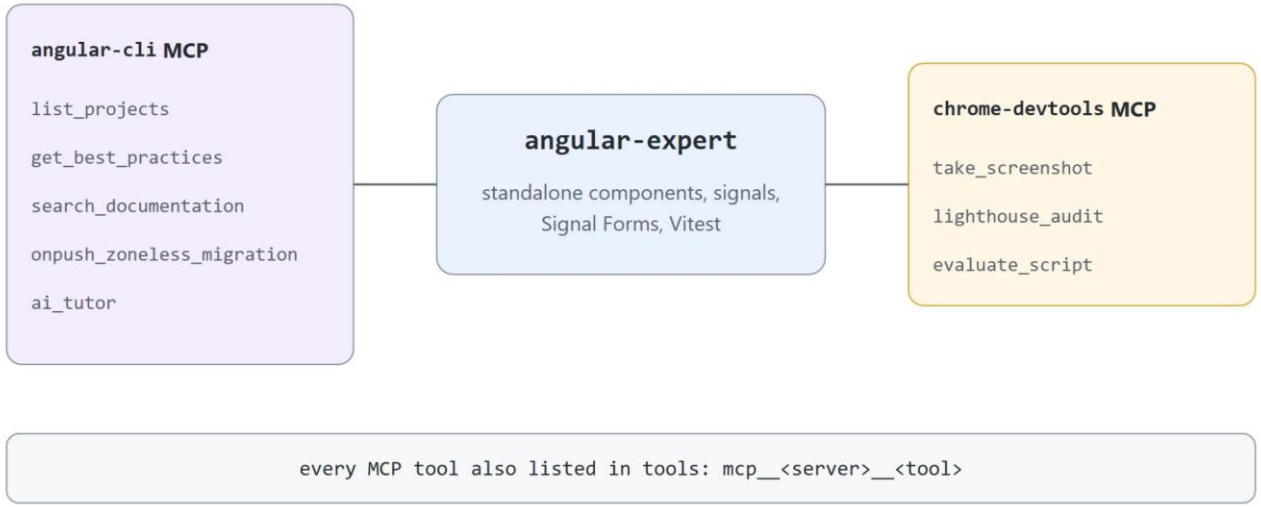
- Decide the **route**, the **signature** and the persistence detail before spawning
- Hand **dotnet-expert** a closed task that names the files to read
- The demo adds `GET /food/search?name=` to `catalog-service-cs`
- Verify with **curl** against the running API, negative cases included



A frontend agent for Angular

angular-expert with the Angular CLI and Chrome DevTools MCPs

- Scoped to Angular v21+: standalone components, signals, Signal Forms, Vitest
- **`angular-cli`** MCP: projects, best practices, docs and zoneless migration
- **`chrome-devtools`** MCP: screenshots, Lighthouse and live DOM checks
- Every MCP tool appears in `tools` as **`mcp__<server>__<tool>`**
- No `permissions` field: command rules stay in `settings.json`



Backend and frontend in parallel

Three specialists, one merged result

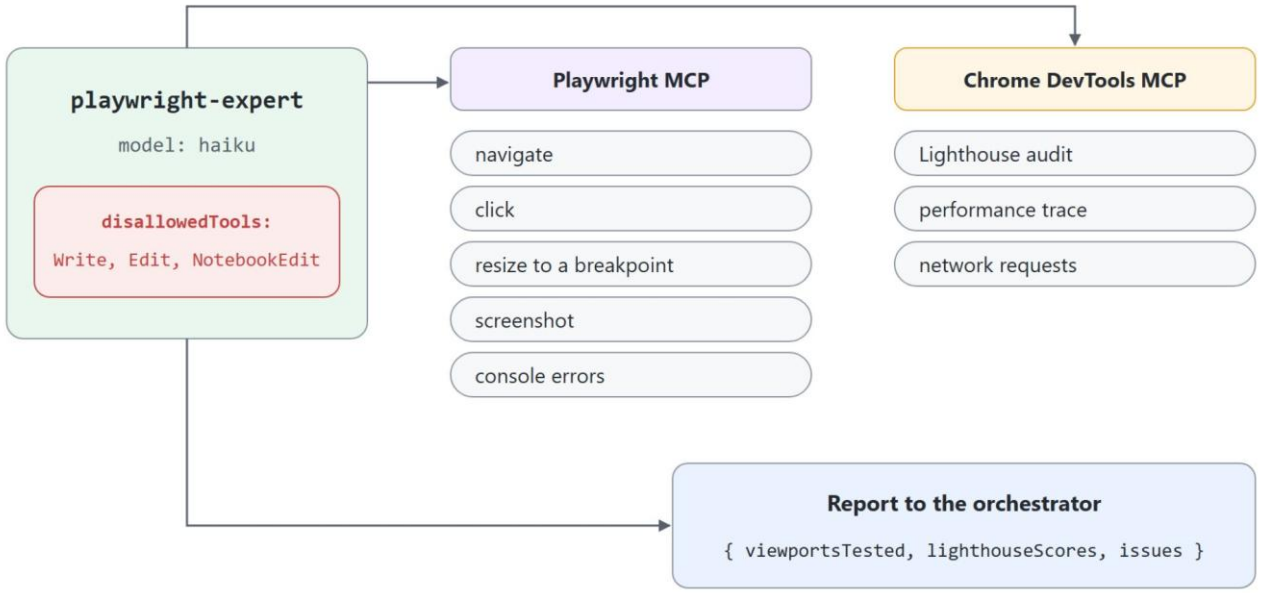
- The orchestrator spawns **dotnet-expert**, **angular-expert** and a testing agent at once
- Each gets one task, its own paths and the same return shape
- `{ status, filesChanged, summary }` from every agent
- The orchestrator waits for all three, then merges the changed files



A QA agent without write access

It tests, audits and reports, and never fixes

- No implementation authority: `disallowedTools: [Write, Edit, NotebookEdit]`
- **Playwright** MCP: navigate, click, resize, screenshot, console errors
- **Chrome DevTools** MCP: Lighthouse, performance traces, network requests
- Runs on **Haiku** for repetitive screenshot and audit loops
- Issues go back to the orchestrator, which re-spawns the implementing agent



Unit tests from a permission-locked agent

The agent reports the defect it may not fix

- The boundary has two halves: the enforced `tools` allowlist and a written **scope** section
- `Write` and `Edit` carry no path filter, so the scope section says where it may write
- **catalog-test-agent** writes and runs xUnit tests for `FoodController`
- Its suite documents the un-awaited `GetById` call inside `DeleteFood`



03

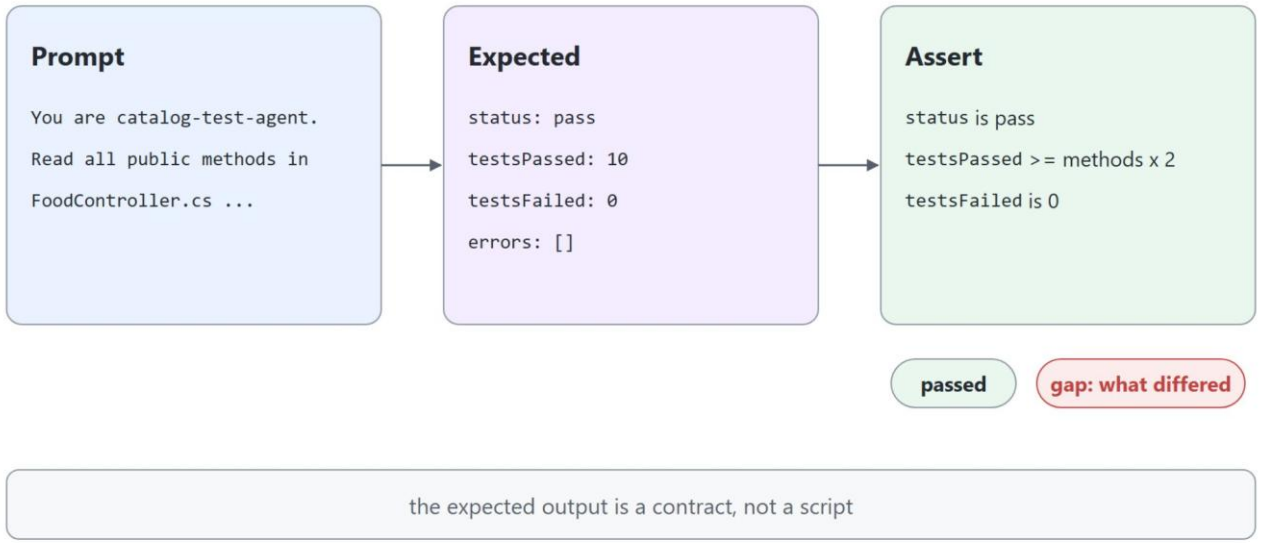
Testing Agent Performance

Test cases, assertions and gaps

Test cases for an agent

Prompt, expected output and assertion

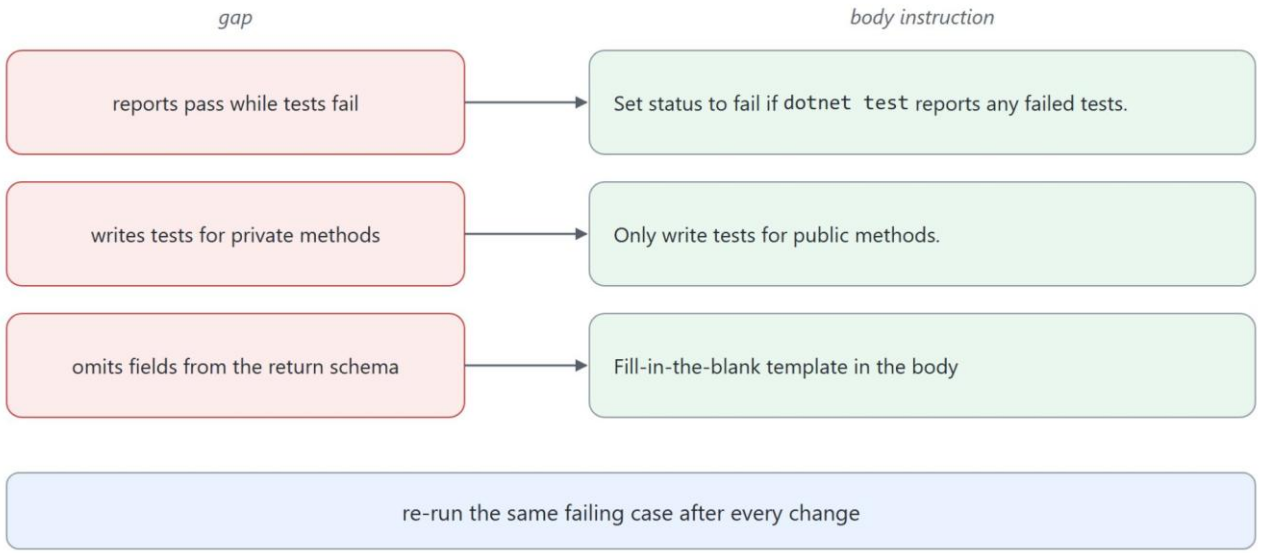
- An agent is a **function**: given a task prompt, it produces an output
- Each test case: one real **prompt**, a precise **expected** output, an **assertion**
- The expected output is a contract, not a script: more tests still pass
- Edge cases matter: a file with no public methods must yield **no tests**
- A failed case returns a **gap** that names the missing behavior



Reading failures as signals

Each gap becomes one instruction in the agent body

- A failed test points to one missing **body instruction**
- Passes while tests fail: it ignores the runner output
- Tests private methods: it is missing a **scope** rule
- Drops fields: it needs an explicit **template**
- Re-run the same case; a new failure names the next instruction



04

Orchestration Patterns

Delegation, loops and graphs

Delegation patterns

The dependency shape picks the pattern

- **Fan-out** for independent work, **prompt chaining** when each step needs the last
- **Hub-and-spoke**: every failure is caught in one coordinator
- The coordinator owns the context: sub-agents see only their prompt
- Fetch shared data **once**, inject the same snapshot into every agent
- Fix the shared interface **before** the fan-out, never alongside it

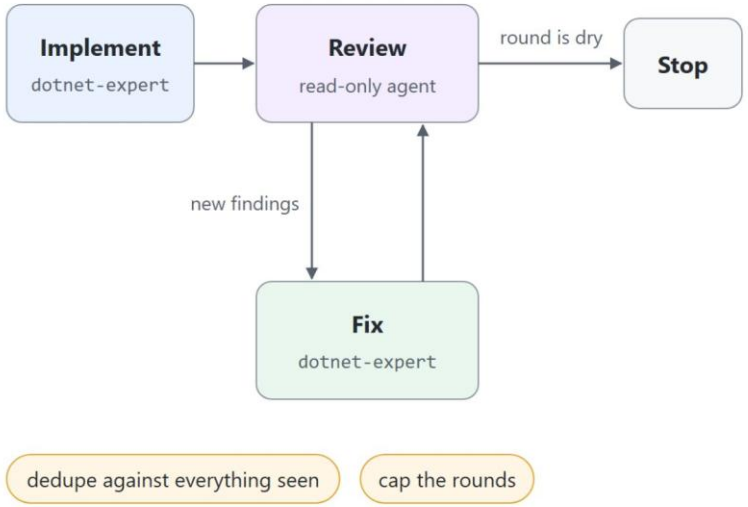
Dependency shape	Pattern	Wrong choice
B needs A, C needs B	**Prompt chaining**	parallel B fabricates its input
subtasks share no data	**Fan-out**	chaining serialises for nothing
any stage can fail	**Hub-and-spoke**	direct handoff hangs or hallucinates
mostly routine, few deep cases	**Pipeline with escalation**	deep path on everything is expensive

the coordinator fetches shared data once and projects it into every prompt

Loops with a stopping rule

Retry on a broken contract, converge on unknown work

- **Retry loop**: repeat the task with the exact validation error attached
- **Convergent loop**: implement, review, fix, review again until a round is dry
- A fixed count is the wrong stopping rule for discovery
- Dedupe against everything **seen**, not only what was confirmed
- Give the reviewer **no write tools**, and cap the rounds



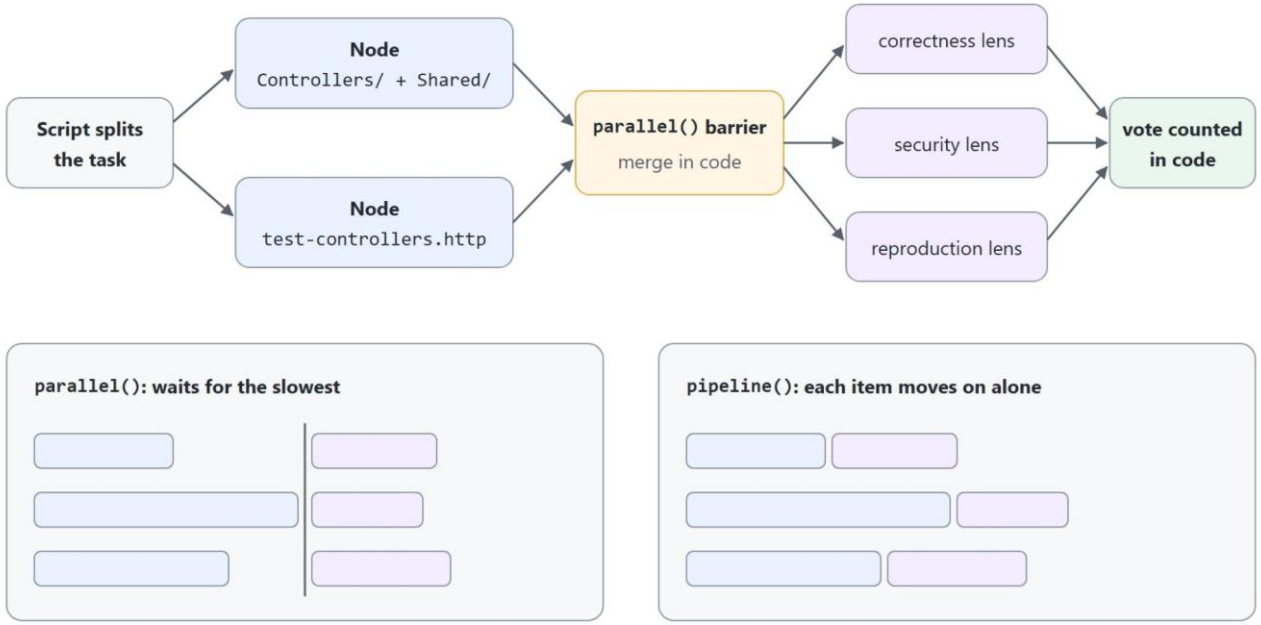
Retry loop

```
previous response failed validation:
filesChanged must be an array
return the corrected JSON only
```

Workflow graphs

The plan moves from a prompt into a script

- Nodes are agent calls, edges are the **data** between them
- A **schema** on each node turns the contract into a checked result
- `parallel()` is a barrier; `pipeline()` streams items stage by stage
- Default to `pipeline()`; a barrier needs the whole set at once
- Put a **verifier** on every edge whose output reaches a human



Sub-agent takeaways

What to carry into agent teams

- Split work when context, serial waiting or role confusion becomes the limit
- Restrict through **tools** and **permissionMode**, not through the prompt
- Give every sub-agent a closed task and a **JSON return contract**
- Test an agent with **cases and assertions**, then fix the body
- Let the **dependency shape** choose delegation, a loop or a graph



05

Agent Teams

Lead, teammates, task list and mailbox

Agent teams in Claude Code

Independent sessions that share a task list and talk to each other

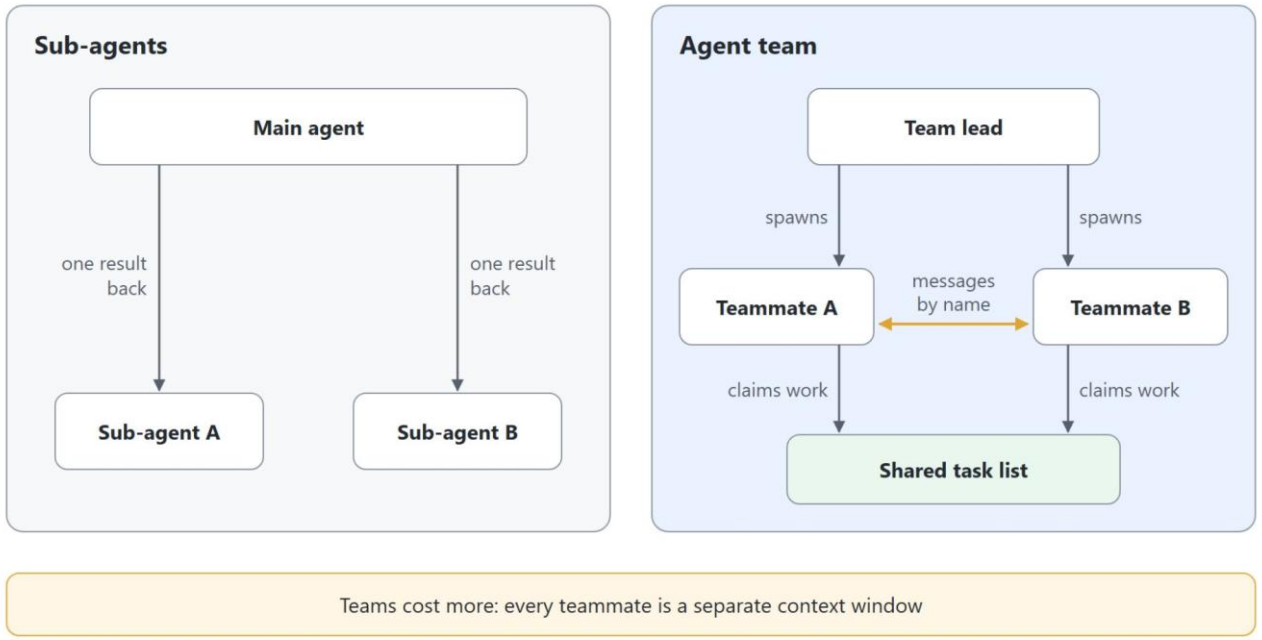
- **Team shape**: a lead, teammates, a shared task list and a mailbox
- **Steering**: spawn prompts, models, the agent panel and direct messages
- **Control**: competing hypotheses, plan approval, hook gates and team size



Sub-agents and agent teams

Whether the workers can reach each other

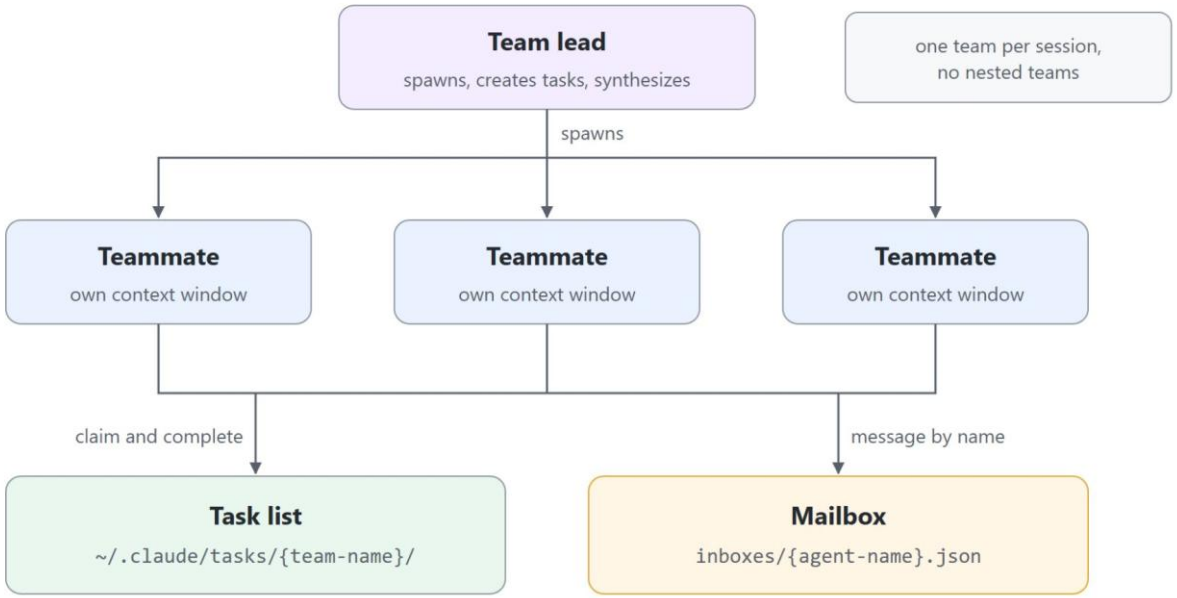
- Sub-agents report one result back to the caller and never hear each other
- Teammates **message each other by name** and coordinate through a **shared task list**
- Teams cost more tokens: each teammate is a separate Claude instance
- Sequential work, same-file edits and long dependency chains get worse in a team



The four parts of a team

Lead, teammates, task list and mailbox

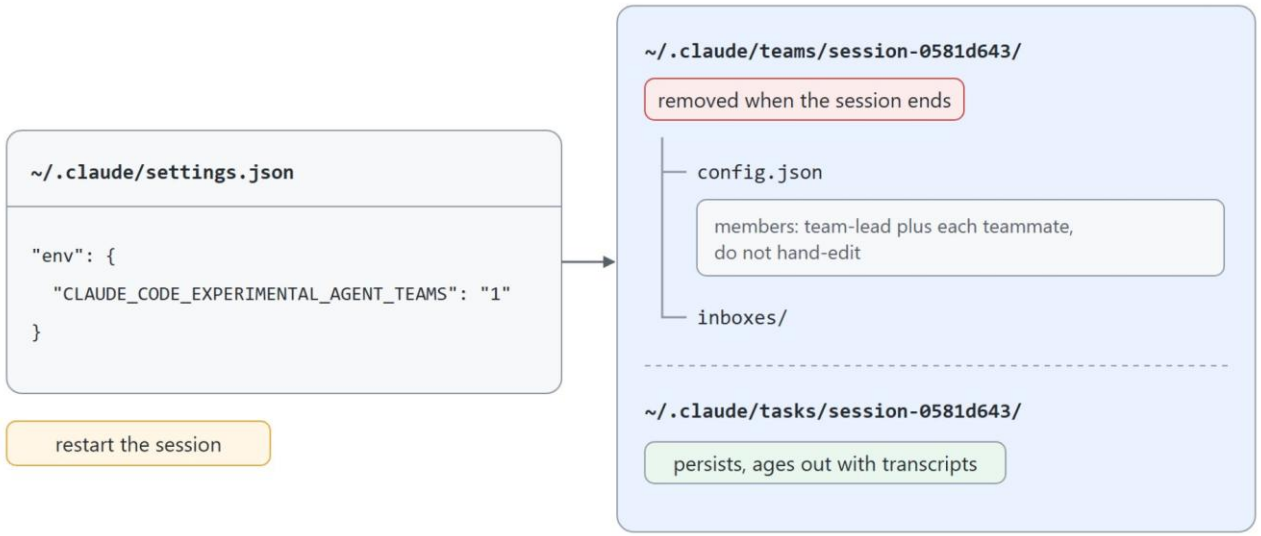
- **Team lead**: the session you started; spawns teammates, creates tasks, merges results
- **Teammates**: separate Claude Code instances working assigned or self-claimed tasks
- **Task list**: the shared work items every agent can see, claim and complete
- **Mailbox**: one inbox per agent, so agents address each other by name
- The lead stays the lead: no promotion, and a teammate cannot spawn teammates



Turning teams on

The experimental flag and what the session writes to disk

- Off by default: set `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` to `1` in `~/.claude/settings.json`
- Read at startup, so restart the session after setting it
- `~/.claude/teams/session-<id>/` holds `config.json` and one inbox per agent
- `~/.claude/tasks/` keeps the task list, so a resumed session keeps its tasks
- Never hand-edit `config.json`: the next state update overwrites it



Limits of agent teams

What to know before you build on them

- One team per session, and no nested teams
- `/resume` and `/rewind` do not bring back in-process teammates
- A task left unmarked blocks every task that depends on it
- Teammates start with the **lead's permission mode**
- A finished teammate reports **idle**, not done: ask for the report on a named channel



06

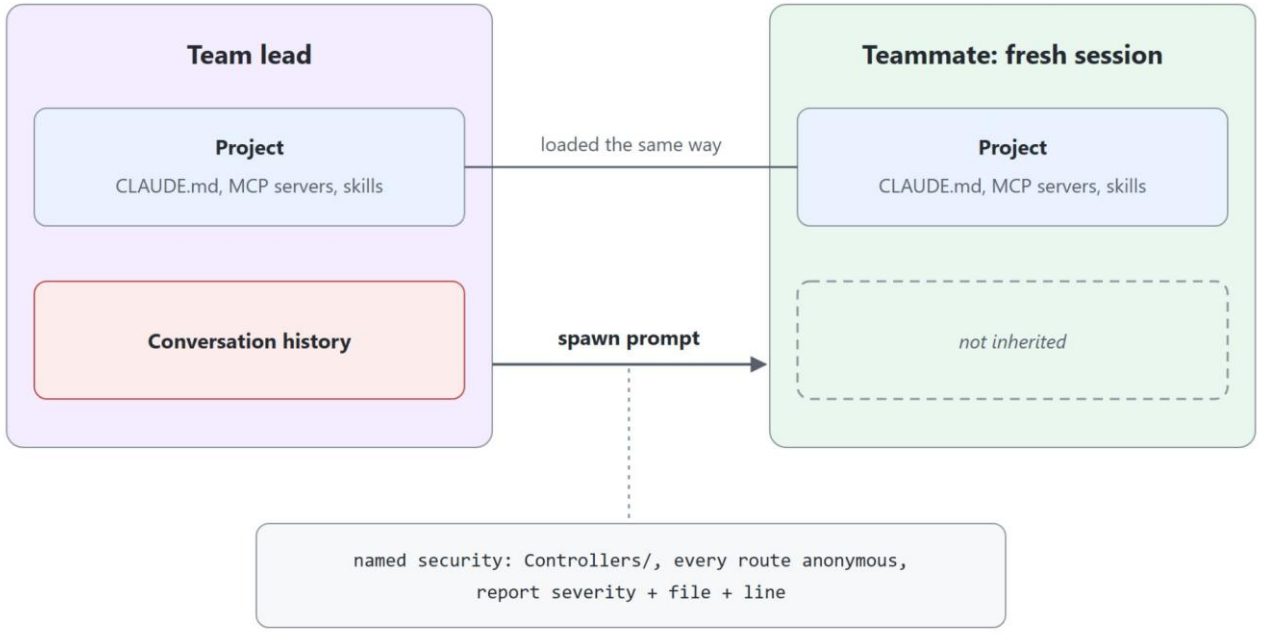
Parallel Review

Spawn, steer and merge three reviewers

Writing the spawn prompt

A teammate inherits your project, not your conversation

- You ask for teammates, or Claude proposes them; nothing spawns without your approval
- A teammate loads `CLAUDE.md`, MCP servers and skills, but **not the lead's conversation**
- Put the job in the prompt: files, stack, what to report and in what shape
- **Name** every teammate, so later instructions resolve to the right session
- Got sub-agents instead? Ask again and say **agent team**



Models and the agent panel

Choosing teammate models and steering from the panel

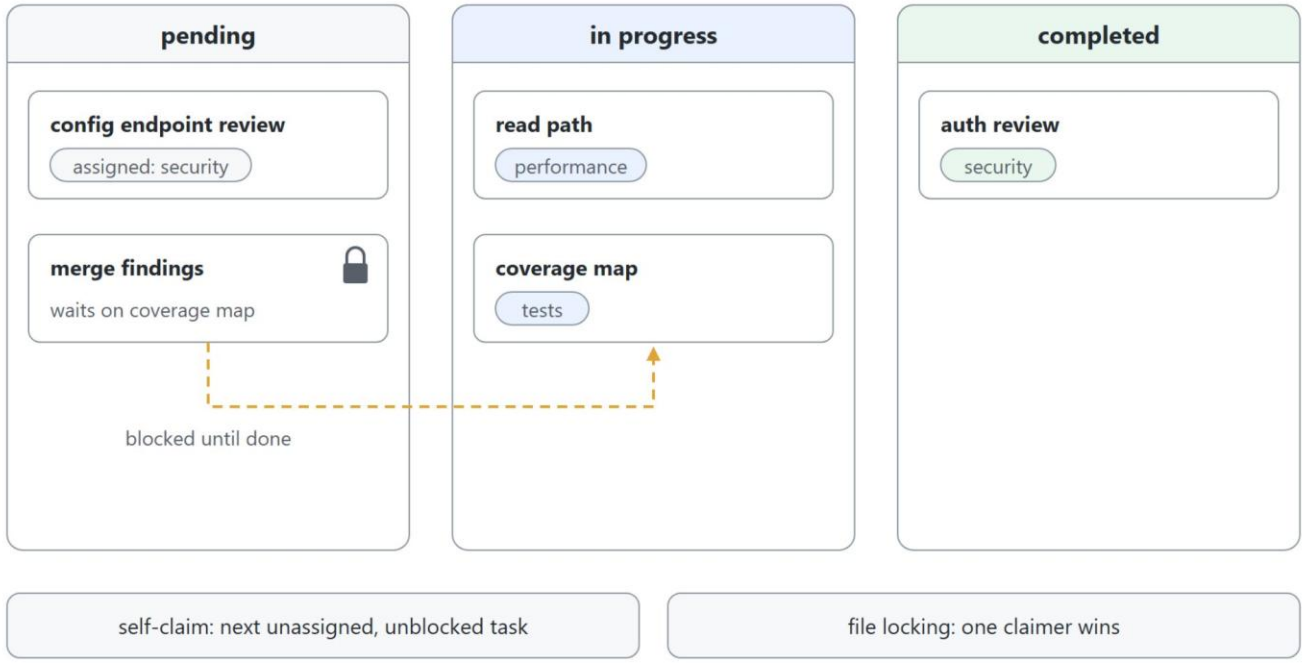
- Teammates take the default from `/config`, not the lead's `/model`; say "Use Sonnet" per spawn
- **Effort** is inherited; model and fast mode are fixed at spawn
- In-process mode runs in any terminal; split panes need tmux or iTerm2
- Enter opens a teammate to message it, Esc interrupts, `x` stops, Ctrl+T shows tasks
- A hidden row has not stopped: idle rows hide after 30 seconds



The shared task list

States, dependencies and claiming

- Three states: **pending**, **in progress**, **completed**
- A task can depend on others; completing the blocker unblocks it
- Work arrives by **assignment** from the lead or by **self-claim** of the next free task
- File locking stops two teammates claiming the same task
- If the lead starts doing the work itself, tell it to wait for its teammates



Three reviewers, one ranked review

The parallel review of catalog-service-cs

- One **lens** per reviewer: security, performance and tests
- Redirect one reviewer mid-run; the other two keep going
- Where two lenses hit the same line, the reviewers settle it by message
- `review-findings.md`: one table ranked by severity, plus a contested section
- The lead reports each teammate as it lands, without polling



07

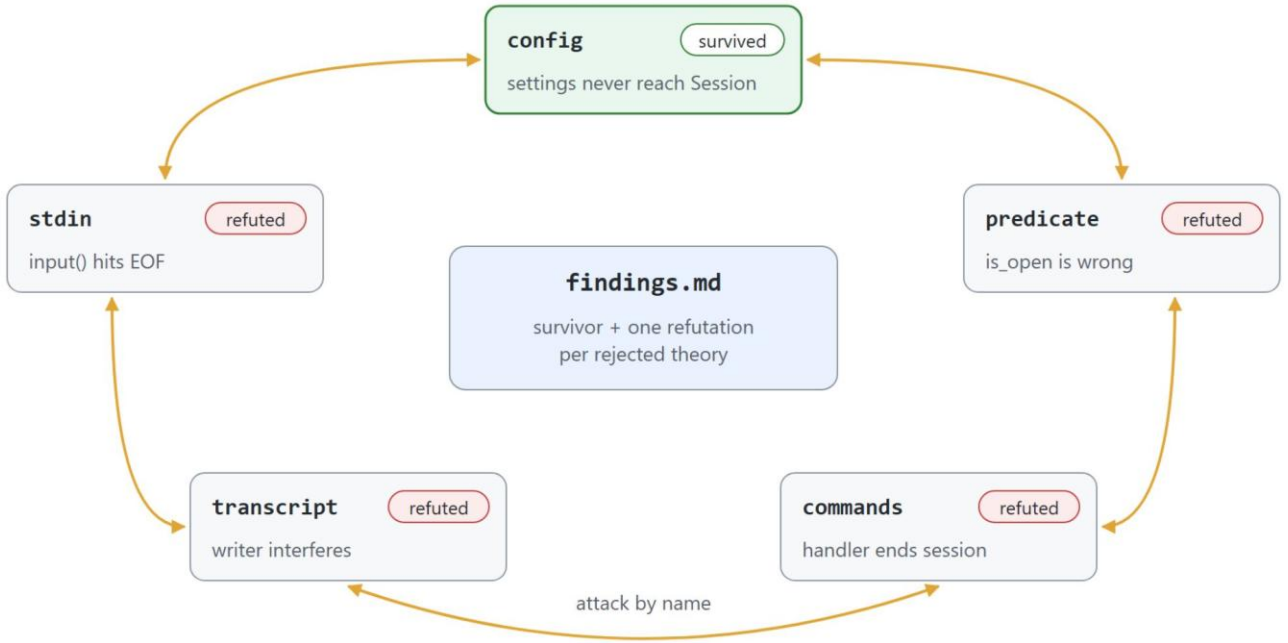
Debate and Control

Competing hypotheses, approvals and gates

Competing hypotheses

Five teammates argue a root cause

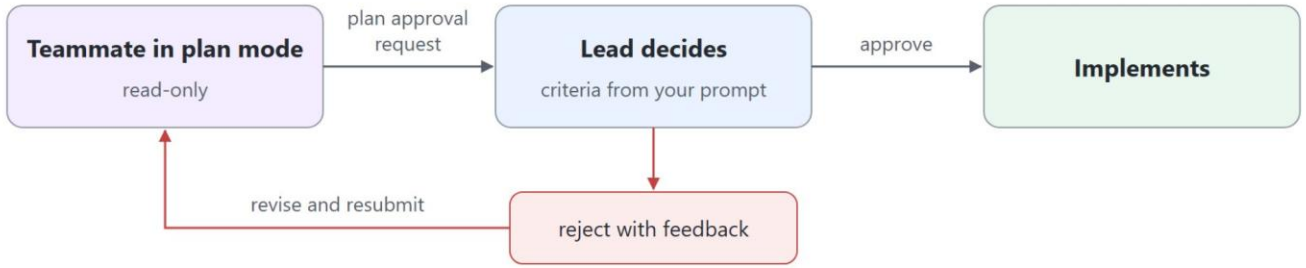
- One investigator **anchors** on the first plausible theory
- Give each teammate one hypothesis and tell them to **disprove each other**
- Each one runs a single-variable control outside the working folder
- The survivor: `maxTurns` in JSON never reaches the `max_turns` setting
- Output is one shared **`findings.md`** with every refutation



Plan approval and team hooks

Controls before and after a teammate works

- **Plan approval**: the teammate stays read-only until the lead approves its plan
- The lead decides on its own, so put the criteria in your prompt
- `TeammateIdle`, `TaskCreated`, `TaskCompleted` block on **exit code 2**
- A `TaskCompleted` gate refuses "done" until the findings file exists
- Do not build gates on the deprecated `team_name` field



`TeammateIdle`	keeps the teammate working
`TaskCreated`	rolls the task back
`TaskCompleted`	prevents the completion

exit 2: stderr returns as feedback

Roles, permissions and team size

Reuse definitions, keep the boundary, start small

- Spawn a teammate from any **sub-agent definition**: its `tools` and `model` apply
- Its `skills` and `mcpServers` do not; `SendMessage` and task tools are always there
- A message from another agent is **not your approval**
- Start with **three to five** teammates and five or six tasks each
- No two teammates own the same file



Takeaways

What to carry into the next module

- Use a team when workers must **disagree**; use sub-agents when only the result matters
- Put everything a teammate needs in the **spawn prompt** and name every teammate
- Split by lens or hypothesis, never two teammates on one file
- Gate with **plan approval** and `TaskCompleted` or `TeammateIdle` hooks
- Start with three to five teammates and read `/cost` before growing

